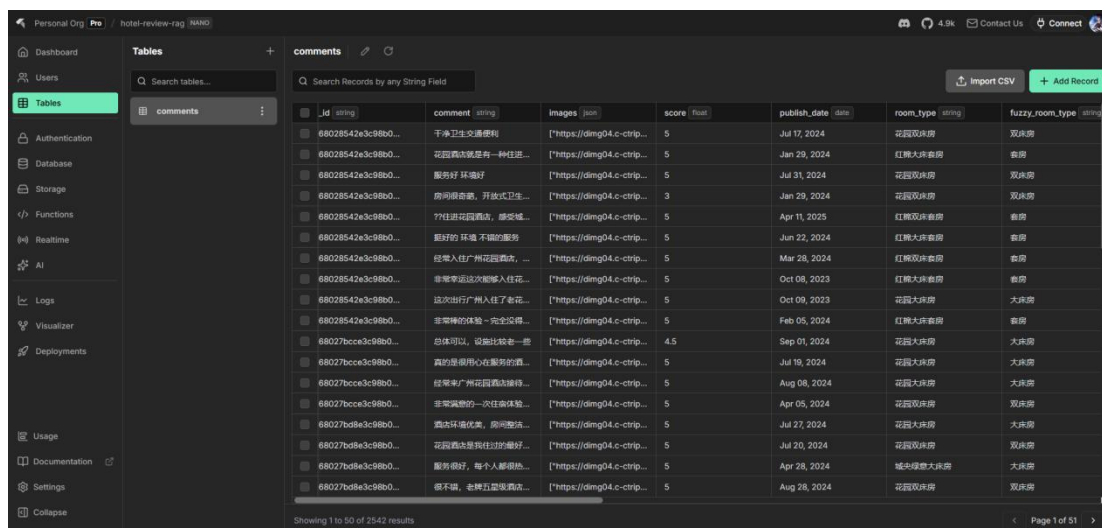


实验 2 作业要求

1. 基于 Next.js 前端框架与 Insforge 后端，使用实验 1 得到的数据或 eLearning 上下载的 enriched_comments.csv 数据，利用 Spec Coding 开发一个全栈 Web 应用

选做：探索 1-2 个好用的 Skills（如前端优化 Skill），辅助项目开发

Insforge 导入数据参考：



The screenshot shows the Insforge web application interface. On the left is a sidebar with navigation links: Dashboard, Users, Tables, Authentication, Database, Storage, Functions, Realtime, AI, Logs, Visualizer, Deployments, Usage, Documentation, Settings, and Collapse. The main area displays a table named 'comments'. The table has columns: _id, comment, images, score, publish_date, room_type, and fuzzy_room_type. The table contains 25 rows of data, showing various user comments and their associated scores and dates.

travel_type	comment_len	log_comment_len	useful_count	log_useful_count	review_count	log_review_count
家庭亲子	8	2.1972246	0	0	8	2.1972246
独自旅行	108	4.691348	0	0	21	3.0910425

quality_score	categories	category1	category2	category3	star
4	["卫生状况", "交通便利性"]	卫生状况	交通便利性	null	5
8	["公共设施", "餐饮设施", "..."]	公共设施	餐饮设施	整体满意度	5

注意：

- ① 建议字段名、字段类型与截图保持一致（将 score 映射为 1-5 的整数 star）
- ② 需将 categories 字段拆分为三个字段，否则 SQL 查询容易出错

2. 思考题 2

完成下方 6 道题目，挑选感兴趣的小题或课堂中提到的其他内容，写写自己的思考，下一节课上可能进行讨论（其他题目的答案不用提交）

3. 提交内容汇总

- （1）项目目录截图、网页截图，提交到“实验课作业”
- （2）个人思考（文档内写上姓名），提交到“独立思考/实验思考题”

4. 截止时间：4/13 23:59

思考题 2

1. 尝试与大模型对话，在 Few-shot Prompting 中，若我们给模型提供了几个格式完全正确但标签全错的示例（如情感分类任务中把所有负面评论都标记为“积极”，正面标记为“消极”），大模型的预测性能会断崖式下降吗？为什么？模型到底从这些示例中“学”到了什么？
2. CoT 通过要求模型输出中间推理步骤，大幅提升了复杂任务的准确率。有人认为这是因为它模拟了人类的逻辑链条，但也有观点认为这仅仅是因为“生成了更多的 Token，争取了更多算力”。假设我们在模型给出最终答案以前，不让它输出逻辑步骤，而是强制它输出一串没有任何意义的填充词（比如“.....”），模型的准确率会有所提升吗？从条件概率的角度，如何理解 CoT 的作用？
3. 为什么在 Prompt 开头加上“你是一个拥有 20 年经验的资深程序员”，“如果回答错误你将被扣除奖金”，往往能显著提高模型的输出质量和逻辑严密性？请跳出拟人化的视角，尝试从预训练数据的分布特征来解释这一现象。
4. 大模型在处理超长上下文时普遍存在 “Lost in the Middle” 现象——对开头和结尾的信息记忆深刻，容易遗忘中间的内容。然而，Transformer 的核心机制 Self-Attention 在数学上是全局对称的。既然机制是对称的，为什么还会产生这种严重的“位置偏见”？
5. 在超长多轮对话中，上下文长度迟早会突破极限。最简单的做法是“滑动窗口”，即直接丢弃更早的记录，但是研究发现如果直接丢弃最开头的几个 Token（即使它们只是无关紧要的打招呼），模型的生成能力会瞬间崩溃，这是为什么？
6. 从 23 年的 “prompt engineering” 到去年的 “context engineering” 再到最近的 “harness engineering”，三者的区别是什么？这反映了一种怎样的趋势演进？